

초등부 1. 수 고르기

풀이 작성자: 박현민

부분문제 1 ($N \leq 3$)

경우의 수가 많지 않아, 각 형태를 모두 조건으로 분리하여 $O(1)$ 에 처리할 수 있다.

부분문제 2 ($N \leq 20$)

$O(2^N)$ 으로 N 개 중 K 개의 수를 고르는 모든 방법을 찾고, 각각 전체 점수를 $O(K^2)$ 또는 $O(N^2)$ 으로 계산하고 그 중 최댓값을 찾으면 된다. $O(2^N N^2)$ 에 해결할 수 있다.

부분문제 3 ($K = 1$)

주어지는 N 개의 수 중 최댓값을 찾으면 된다. $O(N)$ 으로 충분하다.

부분문제 4 ($K = 2$)

수 2개를 $O(N^2)$ 으로 고르고, 각각 전체 점수를 계산하고 그 중 최댓값을 찾으면 된다.

부분문제 5 (주어지는 N 개의 수가 단조증가로 정렬되어 있다.)

각 수의 점수를 '자신의 수'와 '자신의 왼쪽에 있는 수 중 선택된 수의 개수'로 분리해보자. 이 때, '자신의 수'는 어떤 수를 선택하느냐에 따라 달라지고, '자신의 왼쪽에 있는 수 중 선택된 수의 개수'는 해당 수가 선택된 수들 중 왼쪽에서 몇 번째에 있느냐에 따라서 결정된다. 즉, 선택된 수들 중 가장 왼쪽에 있는 수는 그 값이 0이고, 두 번째인 수는 1이고, 세 번째인 수는 2인 것처럼, 오른쪽으로 갈 때마다 1씩 증가하는 규칙을 가진다.

전체 점수를 계산할 때에도 역시 분리해보면 ('자신의 수'의 합)과 ('자신의 왼쪽에 있는 수 중 선택된 수의 개수'의 합)으로 나타낼 수 있는데, 이 때 두 번째 괄호를 살펴보면 각 값은 언급한 것처럼 선택된 수들 중 가장 왼쪽에 있는 수부터 $0, 1, 2, \dots, N-1$ 이다. 즉 두 번째 괄호를 계산하면 언제나 $\frac{N(N-1)}{2}$ 이다.

이제 첫 번째 괄호로 다시 돌아오면, 이제 다른 수와 관계가 없으므로 '자신의 수'가 각자 커질 수 있는 한 최대한 커지는 것이 좋을 수 있다. 주어지는 N 개의 수가 단조증가로 정렬되어 있으므로, 가장 오른쪽 수부터 K 개를 고르면 그 것이 가장 첫 번째 괄호의 값을 크게 하는 방법이다.

부분문제 6

부분문제 5를 해결했다면, 주어지는 N 개의 수를 $O(N^2)$ 이나 $O(N \log N)$ 으로 정렬한 후 동일한 아이디어를 사용할 수 있다.

초등부 2, 중등부 1. 종이접기

풀이 작성자: 이재웅

부분문제 1 ($k = 1$)

직접 모든 경우의 수를 시도해 본 다음 가능한 모든 입력에 대해 답을 저장한 후 출력해주면 된다.

부분문제 2 (U, R 만 존재)

가로와 세로로 접는 방법이 고정되어 있을 경우 접는 순서는 중요하지 않다는 것을 관찰할 수 있고, U 와 R 을 각각 k 번 실행할 때 점의 규칙성을 찾아서 출력해주면 된다.

부분문제 3 ($k \leq 8$)

종이를 접은 순서의 역순으로 펼칠 때마다 뚫린 점의 개수가 2배로 늘어나고 새로 추가되는 점의 위치는 접은 방향과 기존 점의 위치에 의해 결정됨을 알 수 있다. 예를 들어, 0번 위치에 뚫린 점이 있는 종이를 오른쪽으로 펼치면 점은 2배로 늘어나는데, 펼쳐진 종이의 왼쪽에는 0번 점이 그대로 있고 오른쪽에는 1번 점이 있게 된다.

기존의 점과 새로 생긴 점은 접은 선을 기준으로 대칭을 이루므로, 2차원 배열 등을 통해 점의 위치를 기록하면 다음 단계에서 점의 위치 또한 구할 수 있게 된다. 기존 점의 위치와 종이를 펼치는 방향에 따라 새로 생기는 점의 위치를 조건문을 통해 미리 구해놓은 다음, 종이를 차례차례 펼쳐가면서 점들의 변화를 저장한 후 최종 상태를 출력해주면 된다.

초등부 3, 중등부 2. 등산 마니아

풀이 작성자: 구재현

부분문제 1 (그래프가 직선)

두 정수 i, j 를 골랐을 경우 ($i < j$) 다양성이 $j - 1$ 이다. j 를 고정할 경우, 가능한 i 의 개수는 $j - 1$ 이다. 고로 모든 $1 \leq j \leq N$ 에 대해 $(j - 1)^2$ 의 합을 계산하고 출력하면 된다.

부분문제 2 (그래프가 성계)

두 정수 i, j 를 골랐을 경우 ($i < j$) 다양성은 $i = 1$ 일 때 1, 아니면 2이다. $i = 1$ 인 쌍은 $N - 1$ 개 있고, 그렇지 않은 쌍은 $\frac{(N-1)(N-2)}{2}$ 개 있다. 고로 답은 $(N - 1)^2$ 이다.

부분문제 3 (그래프가 완전 이진 트리)

문제의 조건이 복잡하지만, 결국 $N = 2^{d+1} - 1$ 인 깊이 d 의 완전 이진 트리가 주어진다는 뜻이다. i 의 깊이, j 의 깊이, 그리고 1번 정점에서 처음으로 이 두 정점이 갈라지는 위치 k 의 깊이, 이 세 가지를 고정하자. (k 를 보통 i, j 의 *Lowest Common Ancestor (LCA)*) 라고 부른다). i 의 깊이와 j 의 깊이 모두가 k 의 깊이보다 큰 경우, 둘 중 하나만 k 의 깊이보다 큰 경우로 케이스를 나눠서, 가능한 i, j, k 의 경우의 수를 2의 지수승 곱의 수로 계산할 수 있다. $d = O(\log N)$ 이니 $O(d^3) = O(\log^3 N)$ 시간에 문제가 해결된다.

부분문제 4 ($N \leq 100$)

모든 가능한 $i < j$ 를 순회하자. $LCA(i, j)$ 를 두 정점 i, j 의 Lowest Common Ancestor라고 하고, $depth[i]$ 를 i 번 정점과 1번 정점의 거리라고 하면, 답은 $depth[i] + depth[j] - depth[LCA(i, j)]$ 가 된다. 두 정점의 LCA는 $O(N)$ 시간에 계산할 수 있다. 전체 문제가 $O(N^3)$ 시간에 해결된다.

부분문제 5 ($N \leq 5000$)

$dist(i, j)$ 를 두 정점 i, j 의 거리라고 정의하면, 고정된 쌍 $i < j$ 에 대해서 다양성은 $\frac{dist(1, i) + dist(i, j) + dist(j, 1)}{2}$ 이다. 모든 두 정점 간의 거리를 $O(N)$ 번의 깊이 우선 탐색으로 계산해 둘 수 있다. 전체 문제가 $O(N^2)$ 시간에 해결된다.

부분문제 6

각 쌍에 대해서 다양성의 합을 센다는 것은, 각 쌍에 대해서 조건을 만족하는 어떤 간선의 수를 센다는 것이다. 반대로, 모든 간선에 대해서, 이 간선이 특정 쌍의 다양성에 기여하는 조건을 파악하고, 이 간선이 다양성에 기여하는 쌍의 개수를 세어주자.

트리의 루트를 1번 정점으로 두자. v 와 v 의 부모를 잇는 간선에 대해서, v 의 서브트리에 i 혹은 j 정점이 있다면, 이 간선은 1번 정점과 두 정점 중 하나를 잇는 경로 상에 있으니 다양성에 기여한다. 둘 다 없다면, 이 간선이 없다 하더라도 1, i, j 번 정점 사이를 오가는 데는 문제가 없고, 다양성에 기여하지 않는다.

고로, v 와 v 의 부모를 잇는 간선에 대해서 문제의 정답은, 전체 트리에서 두 개의 정점을 고르면서 v 의 서브트리에서 최소 하나의 정점을 뽑는 경우의 수와 동일하다. v 의 서브트리에 속하는 정점의 개수를 $O(N)$ 에 전처리 한다면 이 경우의 수는 간단한 수식으로 셀 수 있다. 전체 문제가 $O(N)$ 에 해결된다.

초등부 4. 사탕 돌리기

풀이 작성자: 구재현

부분문제 1 ($Q = 1$)

다음과 같은 그리디 알고리즘을 사용할 수 있다.

- 현재 껑통에 있는 사탕 중 다른 위치로 옮겨져야 하는 사탕을 아무거나 고른다. 그런 것이 없다면 아무거나 고른다.
- 해당 사탕이 목적지에 도달할 때까지 계속 가지고 다닌다.

이러한 그리디 알고리즘을 구현했을 때 원하는 최종 상태가 나왔다면 문제를 해결할 수 있다. 알고리즘의 정당성은 따로 증명하지 않으나, 전체 문제 풀이에서 자연스럽게 따라온다.

부분문제 2 ($N = 2$)

1번 껑통에 있는 2번 사탕의 개수와, 2번 사탕에 있는 1번 사탕의 개수는 동일하다. 이 숫자를 C 라고 하자. 한 번의 사탕 돌리기로 두 사탕을 바꿔주면, C 를 하나씩 줄여줄 수 있다. 고로 $C \leq Q$ 면 답은 1, 아니면 0이다.

부분문제 5

먼저, 최종 상태를 만들 수 있는 최소 작업 수를 Q_{min} 이라고 하면, $Q_{min} + 1, Q_{min} + 2 \dots$ 번의 작업으로도 원하는 상태를 얻을 수 있다. Q_{min} 번의 작업 이후에, 1번 껑통에서 아무 사탕이나 집은 후 한 바퀴 도는 식으로 남은 작업을 처리하면 되기 때문이다. 고로 최소 작업 수인 Q_{min} 을 구한 후 이것이 Q 이하인지를 판별하는 식으로 문제를 해결한다.

어떠한 사탕이 목표로 하는 껑통에 들어가 있지 않을 경우 이 사탕을 어긋난 사탕 이라고 정의하자. 큰 틀에서, 대략 다음과 같은 그리디 알고리즘을 사용할 수 있다.

- 현재 껑통에 있는 사탕 중 다른 위치로 옮겨져야 하는 사탕을 아무거나 고른다.
- 해당 사탕이 목적지에 도달할 때까지 계속 가지고 다닌다.
- 모든 사탕이 제 자리에 갈 때까지 반복한다.

이 알고리즘의 가장 큰 문제점은 현재 껑통에 있는 사탕 중 다른 위치로 옮겨져야 하는 사탕이 없을 수 있다는 것이다. 일단은 윤이 좋게 그러한 일이 절대 일어나지 않는다고 가정한 후 논리를 전개하자.

첫 번째 관찰은, 어긋난 사탕을 제자리에 놓았을 때, 해당 껑통에 어긋난 사탕이 하나 이상 존재하거나, 현재 위치가 1이다. 만약에 현재 위치가 1이 아니면서 어긋난 사탕이 존재하지 않는다고 한다면, 제자리에 놓게 될 사탕을 포함하여 총 $K + 1$ 개의 사탕이 해당 위치에서 어긋나지 않게 되는데, 같은 색을 가진 사탕이 K 개이기 때문에 이는 모순이다. 고로, 사탕을 제 자리로 돌리는 과정은 하나의 사이클을 이룬다. 이를 정리 사이클 이라고 부르자.

두 번째 관찰은, 앞서 한 가정을 유지했을 때 이 알고리즘이 최적이라는 것이다. 어떠한 사탕 x 가 다른 위치 y 로 옮겨지기 위해서는, $x \leq y$ 일 때 최소 $y - x$ 번 꺼내져야 하고, 그렇지 않을 경우 $N - x + y$ 번 꺼내져야 한다. 이 사실은 어떠한 전략을 사용하더라도 피할 수 없으니, 우리가 사탕을 꺼내야 하는 횟수의 최소 한도는 정해져 있다고 할 수 있다. 한편 위 알고리즘은 항상 이 꺼내야 하는 횟수의 최소 한도만큼의 연산을 사용한다. 고로 위

알고리즘은, 나쁜 경우가 없다는 가정 하에, 최적이다.

이제 전체 알고리즘으로 넘어가자. 현재 내가 있는 위치에서 어긋난 사탕이 존재한다면, 우리는 *정리 사이클* 을 한 바퀴 돌면서 남은 사탕들을 최적의 연산으로 정리해 줄 수 있다. 어긋난 사탕이 없지만 다른 위치에 있다면, 해당 위치에서도 정리 사이클을 구해주자. 우리는 고로 사탕들을 여러 개의 최적인 정리 사이클로 분해할 수 있다. 여기서 세 번째 관찰은, 두 개의 정리 사이클을 하나의 정리 사이클로 합쳐줄 수 있다는 것이다. 정리 사이클 A, B 가 있을 때, B 가 사탕을 교환하는 임의의 궤통에서 멈춰서, (A 의 사탕 내려놓기) $\rightarrow$ (B 의 사탕 집고 B 의 정리 사이클 완료) $\rightarrow$ (다시 A 에서 집으려 했던 사탕, 혹은 내려놓은 사탕을 집은 후 A 의 정리 사이클 완료) 의 요령으로 두 사이클을 한번에 처리할 수 있다.

이렇게 되면 우리는 정리 사이클이 1번 궤통에서 사탕을 교환한다는 가정 하에 최적의 알고리즘을 찾았다. 모든 사탕들의 원호상 거리를 모두 더한 후, 이를 N 으로 나누면 된다. 1번 궤통에서 정리 사이클이 사탕을 교환하지 않는다는 것은, 1번 궤통에 1번 사탕만이 존재한다는 것이다. 이 사탕들에 대해서는 낭비를 피할 수가 없으니 답을 1 늘리는 것은 불가피하며, 아무 사탕이나 집고 정리 사이클을 만난 후, 정리 사이클을 처리하고 다시 그 사탕을 1번 궤통에 넣는 식으로 $\frac{\text{거리합}}{N} + 1$ 번 안에 정리가 가능하다. 이제 전체 문제는 입력으로 주어진 사탕에 대해서 거리 합을 구한 후, 단순 수식으로 쉽게 구현할 수 있다. 시간 복잡도 $O(NK)$, 공간 복잡도 $O(1)$ 에 문제가 해결된다.

부분 문제 3은 정리 사이클로 사탕을 분해하는 방법이 유일하여, 임의의 방법으로 분해할 생각을 하지 않아도 괜찮다. 부분 문제 4는 이 풀이와 다른 접근을 사용하는 $O(N^2K)$ 알고리즘을 위해서 주어졌다. 해당 알고리즘에 대한 설명은 생략한다.
